

Phân tích hồi quy (Regression Analysis)

June 15, 2026

1 Phân tích hồi quy

Phân tích hồi quy là một phương pháp thống kê được sử dụng để nghiên cứu mối quan hệ giữa một biến phụ thuộc (biến mục tiêu) và một hoặc nhiều biến độc lập (biến dự đoán). Mục tiêu chính của phân tích hồi quy là dự đoán hoặc mô tả biến phụ thuộc dựa trên các biến độc lập.

Trong phân tích hồi quy, biến phụ thuộc là biến chúng ta muốn dự đoán hoặc hiểu rõ hơn. Các biến độc lập (còn gọi là biến dự đoán) là những yếu tố mà chúng ta cho là có ảnh hưởng đến biến phụ thuộc. Phân tích hồi quy thường được thực hiện bằng cách xác định một mô hình hồi quy để mô tả mối quan hệ giữa các biến.

Sau đó ta thu thập số liệu, phân tích, để cuối cùng ra được phương trình ví dụ như sau:

```
Dự định mua = 0.2 +  
              + 0.3*(chuẩn chủ quan) +  
              + 0.4*(nhận thức kiểm soát hành vi) +  
              + 0.5*(chất lượng cảm nhận) +  
              + 0.6*(giá trị cảm nhận) +  
              + 0.7*(hiểu biết về sản phẩm) +  
              + 0.8*(nhận biết thương hiệu)
```

2 Quy trình cơ bản

Chúng ta có các bước sau: 1. Xác định vấn đề: xác định biến độc lập và biến phụ thuộc 2. Thu thập và làm sạch dữ liệu 3. Phân tích dữ liệu: xác định mức độ phụ thuộc giữa các biến 4. Xây dựng mô hình hồi quy: lựa chọn mô hình hồi quy phù hợp 5. Chạy thử 6. Kiểm tra, đánh giá

3 Nội dung

3.1 Chuẩn bị

Trong bài này, chúng ta sẽ học cách phân tích hồi quy bằng python qua gói scikit-learn. Để cài đặt, có thể chạy câu lệnh sau:

```
pip install scikit-learn -U
```

```
[1]: import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression #hồi quy tuyến tính
from sklearn.datasets import load_diabetes #dữ liệu mẫu
```

```
[2]: #dữ liệu hồi quy tuyến tính một chiều
x = np.random.uniform(0, 10, 100) #tạo 100 điểm ngẫu nhiên từ 0 -> 200
y = 2*x + np.random.normal(0, 2, 100) #y = 2x + thành phần ngẫu nhiên
data = pd.DataFrame({'x': x, 'y': y}).sort_values('x').reset_index(drop = True)
data.head()
```

```
[2]:
```

	x	y
0	0.290388	-2.159582
1	0.294279	0.654210
2	0.309946	-1.965610
3	0.318901	1.087503
4	0.331848	-0.302756

```
[3]: # dữ liệu hồi quy tuyến tính nhiều chiều
diabetes, target = load_diabetes(as_frame = True, return_X_y = True)
d, t = load_diabetes(as_frame = True, return_X_y = True, scaled = False)
```

```
[4]: # dữ liệu đã được chuẩn hóa bằng điểm Z
diabetes.head()
```

```
[4]:
```

	age	sex	bmi	bp	s1	s2	s3	\
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	

	s4	s5	s6
0	-0.002592	0.019907	-0.017646
1	-0.039493	-0.068332	-0.092204
2	-0.002592	0.002861	-0.025930
3	0.034309	0.022688	-0.009362
4	-0.002592	-0.031988	-0.046641

```
[5]: #dữ liệu gốc
d.head()
```

```
[5]:
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	59.0	2.0	32.1	101.0	157.0	93.2	38.0	4.0	4.8598	87.0
1	48.0	1.0	21.6	87.0	183.0	103.2	70.0	3.0	3.8918	69.0

2	72.0	2.0	30.5	93.0	156.0	93.6	41.0	4.0	4.6728	85.0
3	24.0	1.0	25.3	84.0	198.0	131.4	40.0	5.0	4.8903	89.0
4	50.0	1.0	23.0	101.0	192.0	125.4	52.0	4.0	4.2905	80.0

```
[6]: target.head()
```

```
[6]: 0    151.0
     1     75.0
     2    141.0
     3    206.0
     4    135.0
     Name: target, dtype: float64
```

3.2 Cơ sở lý thuyết

3.2.1 Hồi quy tuyến tính một biến

Cho **một** biến độc lập x và **một** biến phụ thuộc y , phân tích hồi quy tuyến tính đơn giản (Simple Linear Regression Analysis) là tìm sự liên hệ giữa 2 biến số theo dạng:

$$y = \alpha x + \beta + \epsilon$$

Trong đó: - α và β được gọi chung là hệ số hồi quy. α còn được gọi là hệ số góc, β còn được gọi là hệ số tự do. - ϵ là sai số hồi quy.

Mục tiêu của hồi quy là tìm α và β sao cho ϵ là **tối thiểu**.

3.2.2 Hồi quy tuyến tính nhiều biến

Cho n biến độc lập $x_1, x_2, \dots, x_n$ và **một** biến phụ thuộc y , phân tích hồi quy tuyến tính nhiều biến là tìm sự liên hệ dưới dạng:

$$y = \alpha_1 x_1 + \alpha_2 x_2 + \dots + \alpha_n x_n + \beta + \epsilon$$

Mục tiêu là tìm $\alpha_1, \alpha_2, \dots, \alpha_n$ và β sao cho ϵ là **tối thiểu**.

3.2.3 Phương pháp

Phương pháp thường được sử dụng là tìm các hệ số hồi quy sao cho tổng bình phương sai số là tối thiểu (OLS, Ordinary Least Squares).

3.2.4 Giả định hồi quy

Các giả định cần thiết trong hồi quy tuyến tính: 1. Có mối quan hệ tuyến tính giữa biến độc lập và biến phụ thuộc. 2. Sai số hồi quy ϵ (hiệu của giá trị quan sát và giá trị ước đoán) không tương quan với biến độc lập x_i . 3. Sai số hồi quy ϵ tuân theo phân phối chuẩn với trung bình là 0. 4. x_i không có sai sót ngẫu nhiên trong quá trình đo lường.

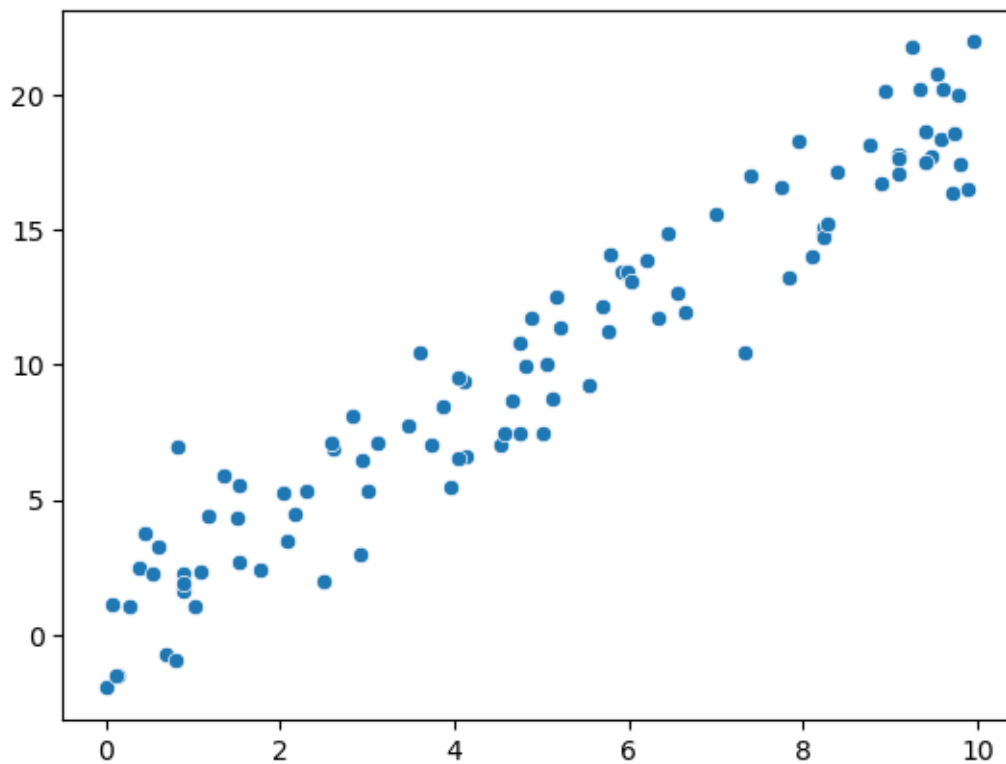
Cách để phát hiện mối tương quan giữa x và y : - Tính hệ số tương quan. - Dùng biểu đồ phân tán (scatter plot).

```
[21]: # tính ma trận tương quan  
np.corrcoef(x, y)
```

```
[21]: array([[1.          , 0.95997684],  
          [0.95997684, 1.          ]])
```

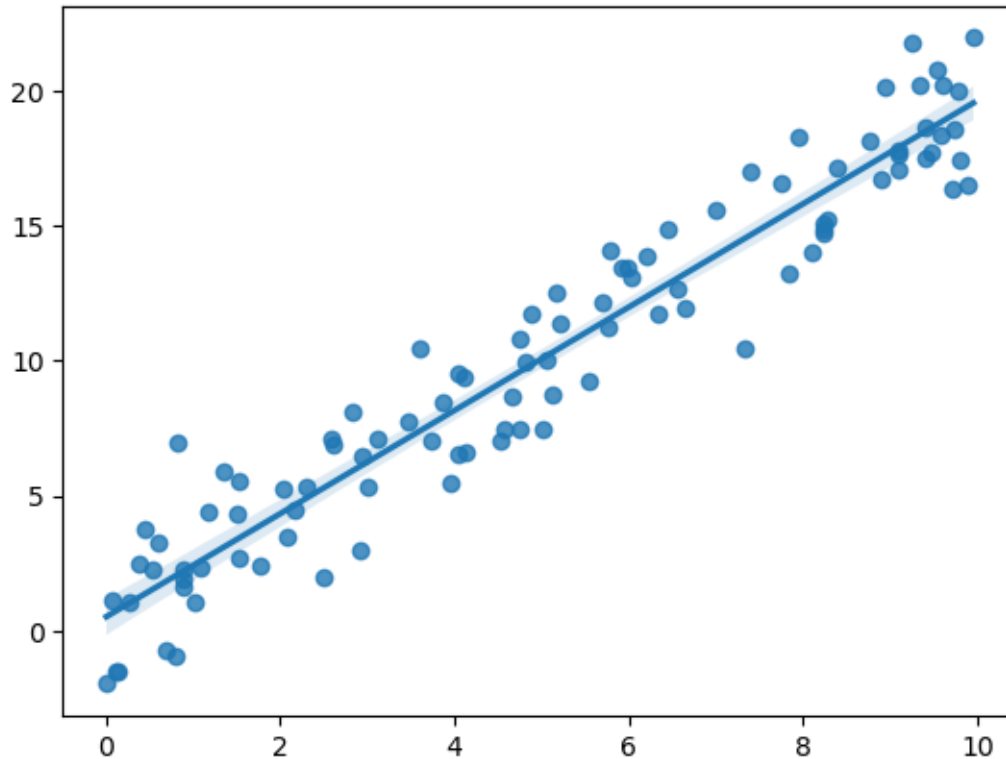
```
[22]: # vẽ biểu đồ phân tán  
sns.scatterplot(x = x, y = y)
```

```
[22]: <Axes: >
```



```
[23]: sns.regplot(x = x, y = y)
```

```
[23]: <Axes: >
```



3.2.5 Đánh giá mô hình hồi quy

Người ta thường dùng hệ số xác định (Coefficient of determination) R^2 để đánh giá tính chính xác của mô hình hồi quy, ta có:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Trong đó: - y là giá trị quan sát. - $\hat{y}$ là giá trị dự đoán từ mô hình. - $\bar{y}$ là giá trị trung bình của giá trị quan sát.

Ta có $0 < R^2 < 1$.

3.3 Sử dụng python

Trong python, có thể dùng đến `LinearRegression` (của thư viện `sklearn`) để tìm các hệ số hồi quy.

Để tìm (học) hệ số hồi quy từ dữ liệu, ta dùng đến phương thức `.fit(x, y)` của `LinearRegression`.

Sau khi học xong, ta có thể dùng đến các thuộc tính và phương thức sau: - `.coef_`: trả về các hệ số góc. - `.intercept_`: trả về hệ số tự do. - `.predict(x)`: dự đoán giá trị của y từ x . - `.score(x, y)`: trả về điểm đánh giá của mô hình.

```
[16]: # hồi quy tuyến tính một biến
      # chia tách dữ liệu
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y)
```

```
[17]: # học  
slr = LinearRegression().fit(x_train.reshape(-1, 1), y_train)
```

```
[18]: # hệ số hồi quy  
a, b = slr.coef_, slr.intercept_  
print(a, b)
```

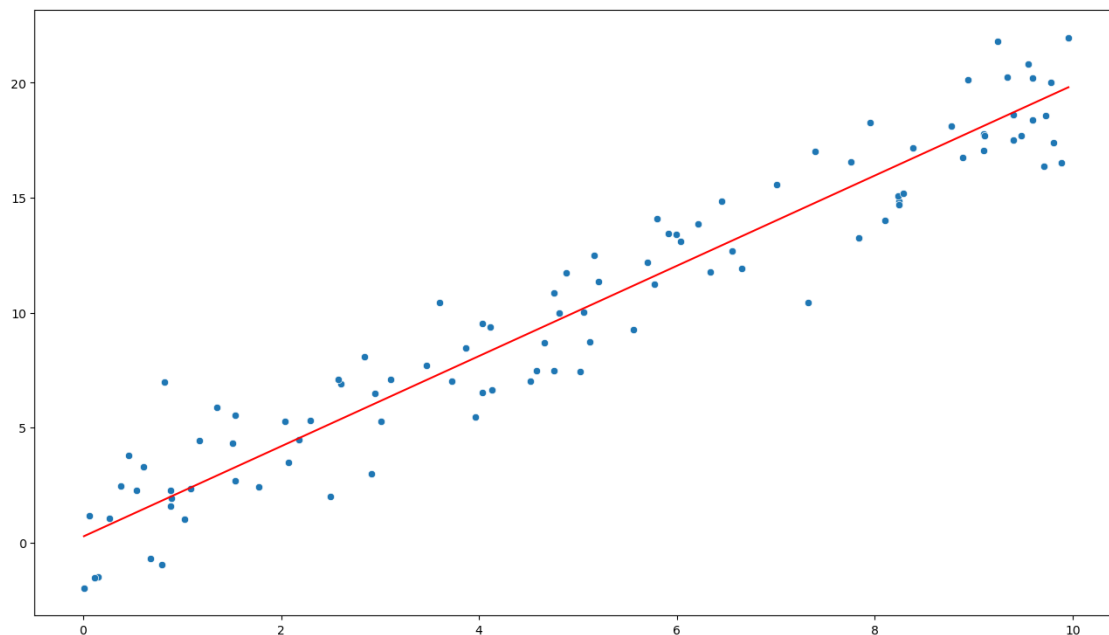
```
[2.04385949] -0.39893191164538955
```

```
[19]: # đánh giá mô hình  
slr.score(x_test.reshape(-1, 1), y_test)
```

```
[19]: 0.8996785695118015
```

```
[41]: import matplotlib.pyplot as plt  
  
fig = plt.figure(figsize = (16, 9))  
ax = fig.add_subplot(1, 1, 1)  
  
sns.scatterplot(x = x, y = y, ax = ax)  
line = np.linspace(x.min(), x.max())  
sns.lineplot(x = line, y = a[0]*line + b, ax = ax, color = 'red')
```

```
[41]: <Axes: >
```



```
[28]: # hồi quy tuyến tính nhiều biến
# chia dữ liệu thành phần học và phần kiểm
d_train, d_test, t_train, t_test = train_test_split(diabetes, target)
```

```
[29]: # học
mlr = LinearRegression().fit(d_train, t_train)
```

```
[30]: # hệ số hồi quy
mlr.coef_, mlr.intercept_
```

```
[30]: (array([ 5.68549202, -287.99682883, 534.4771466 , 321.8766684 ,
           -326.27511918,  81.86825048, -125.97548944, 161.41471773,
            608.03731924,  14.26097423]),
      np.float64(152.48555968706864))
```

```
[31]: # đánh giá mô hình
mlr.score(d_test, t_test)
```

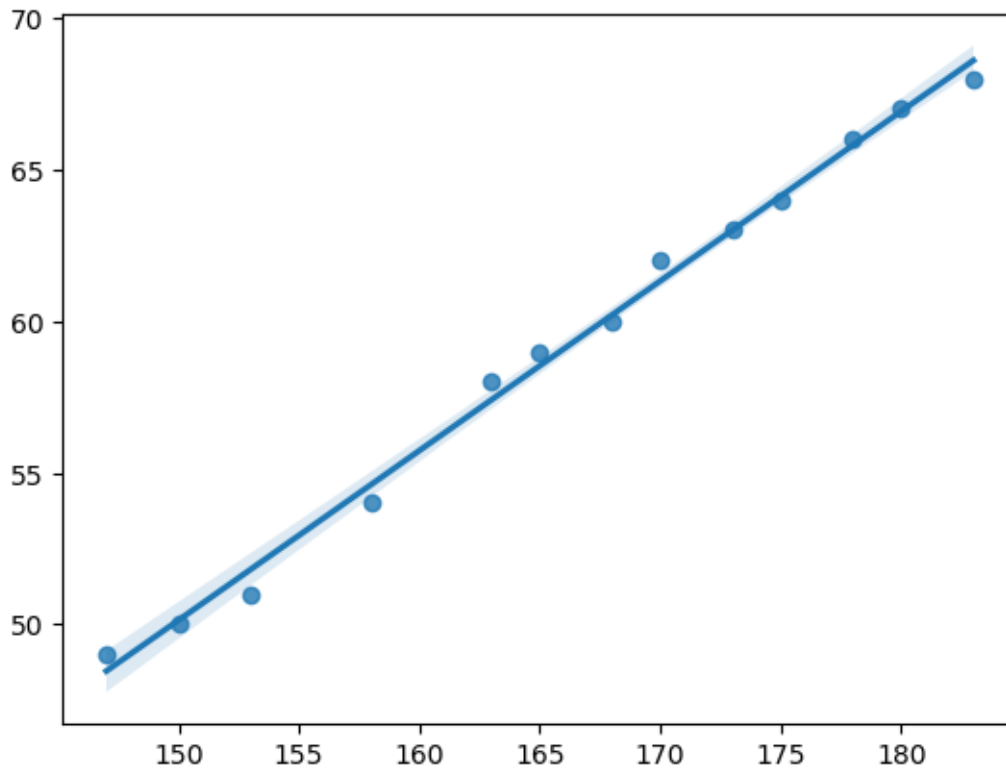
```
[31]: 0.5469172222648362
```

3.4 Hạn chế của hồi quy tuyến tính

Hạn chế đầu tiên của Linear Regression là nó rất nhạy cảm với nhiễu (sensitive to noise).

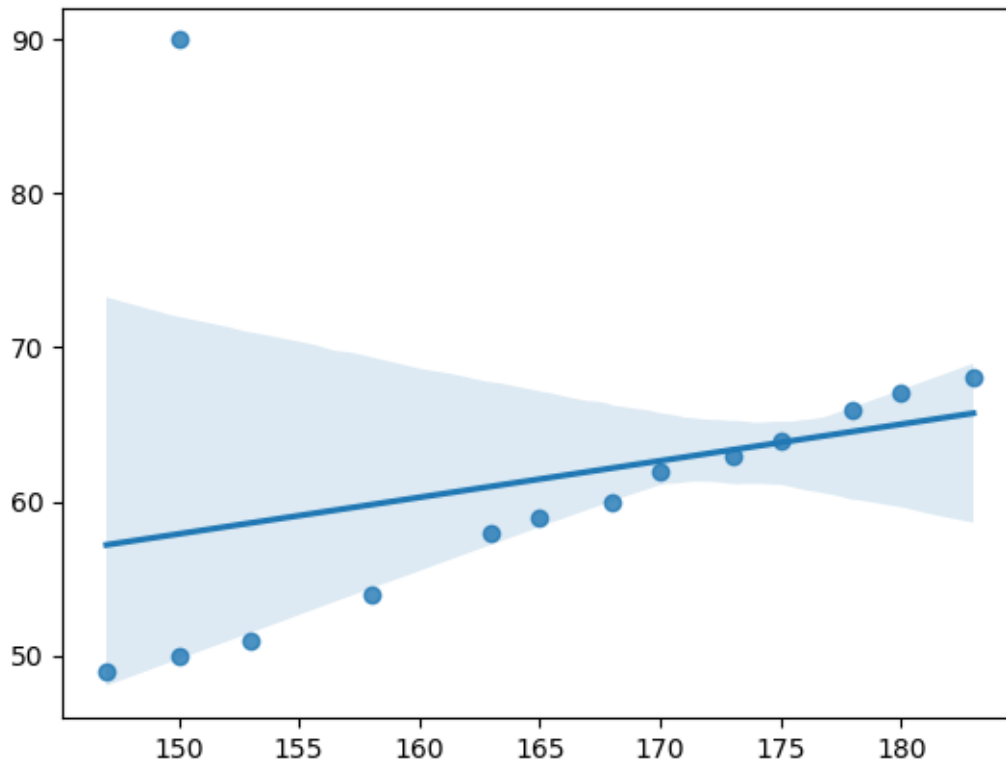
```
[11]: # ví dụ
height = np.array([[147, 150, 153, 158, 163, 165, 168, 170, 173, 175, 178, 180,
                  ↪183]])
weight = np.array([[ 49, 50, 51,  54, 58, 59, 60, 62, 63, 64, 66, 67, 68]])
sns.regplot(x = height, y = weight)
```

```
[11]: <Axes: >
```



```
[12]: height = np.append(height, 150)
      weight = np.append(weight, 90)
      sns.regplot(x = height, y = weight)
```

```
[12]: <Axes: >
```

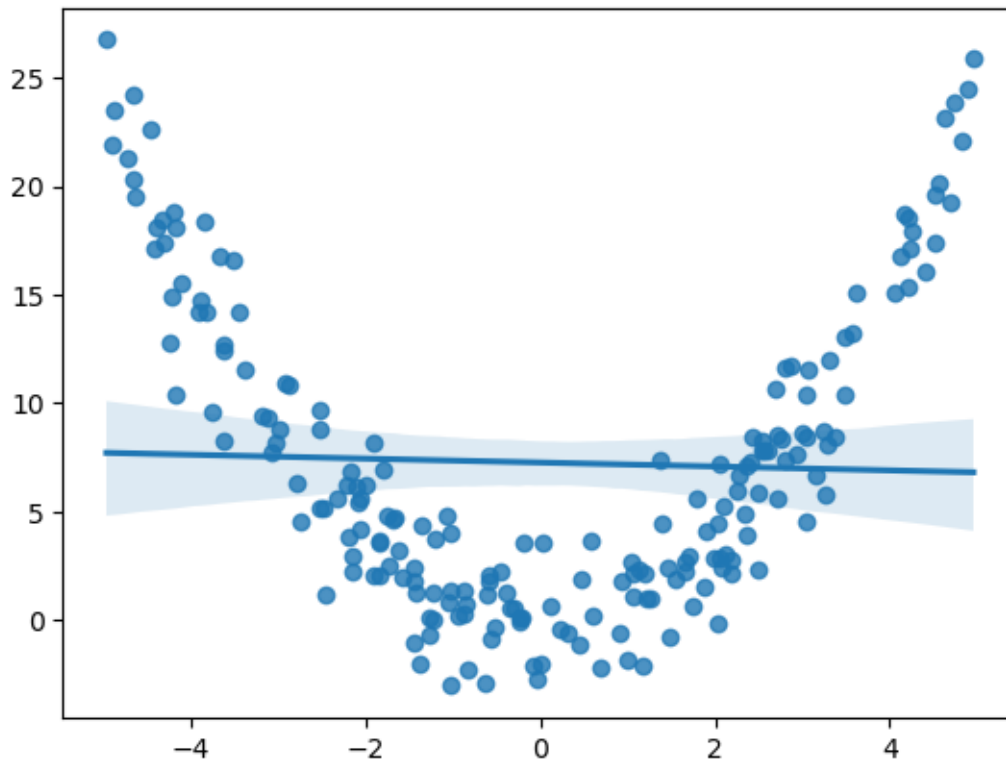



Vì vậy, trước khi thực hiện Linear Regression, các nhiễu (outlier) cần phải được loại bỏ. Bước này được gọi là tiền xử lý (pre-processing).

Hạn chế thứ hai của Linear Regression là nó không biểu diễn được các mô hình phức tạp.

```
[24]: x = np.random.uniform(-5, 5, 200) #tạo 100 điểm ngẫu nhiên từ 0 -> 200
      y = x*x + np.random.normal(0, 2, 200) #y = 2x + thành phần ngẫu nhiên
      sns.regplot(x = x, y = y)
```

```
[24]: <Axes: >
```



```
[26]: np.corrcoef(x, y)
```

```
[26]: array([[ 1.          , -0.03487233],  
            [-0.03487233,  1.          ]])
```